
Artificial Intelligence

BS (CS) Spring 2026

Lab 14 Manual



Learning Objectives:

1. Unsupervised Learning

Lab Manual

Unsupervised Learning:

Unsupervised learning is a type of machine learning where algorithms **find hidden patterns in data** without being given labeled examples or “correct answers” to learn from.

When to Use Unsupervised Learning

- You have data but **no labels**
- You want to **explore and understand** data structure
- You need to **compress or reduce** high-dimensional data
- You want to **detect anomalies** or unusual patterns
- You want to **find associations** between items

Key Categories

Category	Goal	Example Algorithms
Clustering	Group similar data points	K-Means, DBSCAN, Hierarchical
Dimensionality Reduction	Compress features	PCA, t-SNE, UMAP
Generative Models	Learn data distribution	GMM, Autoencoders
Association Rules	Find co-occurrence	Apriori, FP-Growth
Anomaly Detection	Find outliers	Isolation Forest, LOF

2. Environment Setup

Install Required Libraries

```
pip install numpy pandas matplotlib seaborn scikit-learn scipy umap-learn mlxtend
```

Standard Imports Template

```
# Core
import numpy as np
import pandas as pd

# Visualization
import matplotlib.pyplot as plt
import seaborn as sns

# Scikit-learn - clustering
from sklearn.cluster import KMeans, DBSCAN, AgglomerativeClustering
from sklearn.mixture import GaussianMixture

# Scikit-learn - dimensionality reduction
from sklearn.decomposition import PCA
from sklearn.manifold import TSNE

# Scikit-learn - preprocessing & evaluation
from sklearn.preprocessing import StandardScaler, MinMaxScaler
from sklearn.metrics import (
```

```

        silhouette_score, silhouette_samples,
        davies_bouldin_score, calinski_harabasz_score,
        adjusted_rand_score
    )

# Anomaly detection
from sklearn.ensemble import IsolationForest
from sklearn.neighbors import LocalOutlierFactor

# Association rules
from mlxtend.frequent_patterns import apriori, association_rules
from mlxtend.preprocessing import TransactionEncoder

# UMAP (install separately: pip install umap-learn)
import umap

# Suppress warnings
import warnings
warnings.filterwarnings('ignore')

# Global style
plt.style.use('seaborn-v0_8-whitegrid')
np.random.seed(42)

```

3. Clustering Algorithms

3.1 K-Means Clustering

K-Means partitions data into **K clusters** by minimizing the within-cluster sum of squared distances to the centroid.

Algorithm Steps:

1. Randomly initialize K centroids
2. Assign each point to the nearest centroid
3. Recompute centroids as the mean of assigned points
4. Repeat steps 2–3 until convergence

Basic Implementation

```

from sklearn.cluster import KMeans
from sklearn.datasets import make_blobs
from sklearn.preprocessing import StandardScaler
import matplotlib.pyplot as plt

# Generate sample data
X, y_true = make_blobs(n_samples=300, centers=4, cluster_std=0.60,
random_state=42)

# Preprocessing
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# Fit K-Means
kmeans = KMeans(
    n_clusters=4,
    init='k-means++',    # Smart initialization (better than random)
    n_init=10,           # Run 10 times, pick best result
    max_iter=300,
    random_state=42
)

```

```

kmeans.fit(X_scaled)

labels      = kmeans.labels_
centroids   = kmeans.cluster_centers_
inertia     = kmeans.inertia_

print(f"Inertia (WCSS): {inertia:.2f}")
print(f"Silhouette Score: {silhouette_score(X_scaled, labels):.4f}")

# Visualize
plt.figure(figsize=(8, 5))
plt.scatter(X_scaled[:, 0], X_scaled[:, 1], c=labels, cmap='tab10', s=40,
            alpha=0.7)
plt.scatter(centroids[:, 0], centroids[:, 1], c='red', marker='X', s=200,
            zorder=5, label='Centroids')
plt.title('K-Means Clustering')
plt.legend()
plt.tight_layout()
plt.show()

```

Finding Optimal K — The Elbow Method

```

inertias     = []
sil_scores   = []
K_range      = range(2, 11)

for k in K_range:
    km = KMeans(n_clusters=k, init='k-means++', n_init=10, random_state=42)
    km.fit(X_scaled)
    inertias.append(km.inertia_)
    sil_scores.append(silhouette_score(X_scaled, km.labels_))

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 5))

ax1.plot(K_range, inertias, 'bo-')
ax1.set_xlabel('Number of Clusters (K)')
ax1.set_ylabel('Inertia (WCSS)')
ax1.set_title('Elbow Method')

ax2.plot(K_range, sil_scores, 'rs-')
ax2.set_xlabel('Number of Clusters (K)')
ax2.set_ylabel('Silhouette Score')
ax2.set_title('Silhouette Analysis')

plt.tight_layout()
plt.show()

```

Silhouette Plot (Advanced)

```

from sklearn.metrics import silhouette_samples
import matplotlib.cm as cm

def plot_silhouette(X, n_clusters):
    km = KMeans(n_clusters=n_clusters, random_state=42, n_init=10)
    labels = km.fit_predict(X)
    sil_avg = silhouette_score(X, labels)
    sil_vals = silhouette_samples(X, labels)

    fig, ax = plt.subplots(figsize=(8, 5))
    y_lower = 10
    colors = cm.nipy_spectral(np.linspace(0.2, 0.8, n_clusters))

    for i, color in zip(range(n_clusters), colors):
        ith_vals = np.sort(sil_vals[labels == i])

```

```

        size = ith_vals.shape[0]
        y_upper = y_lower + size
        ax.fill_betweenx(np.arange(y_lower, y_upper), 0, ith_vals,
color=color)
        ax.text(-0.05, y_lower + size / 2, str(i))
        y_lower = y_upper + 5

    ax.axvline(sil_avg, color="red", linestyle="--", label=f"Avg =
{sil_avg:.2f}")
    ax.set_title(f"Silhouette Plot (K={n_clusters})")
    ax.set_xlabel("Silhouette Coefficient")
    ax.set_ylabel("Cluster")
    ax.legend()
    plt.tight_layout()
    plt.show()

plot_silhouette(X_scaled, 4)

```

K-Means Limitations:

- Assumes spherical, equally-sized clusters
- Sensitive to outliers and initialization
- Must specify K in advance

3.2 Hierarchical Clustering

Builds a tree (dendrogram) of nested clusters. Does **not** require specifying K in advance.

Linkage Criteria:

Method	Merge by
ward	Minimize within-cluster variance (best general use)
complete	Maximum distance between clusters
average	Average distance
single	Minimum distance (chain effect)

Dendrogram

```

from scipy.cluster.hierarchy import dendrogram, linkage
from sklearn.datasets import make_blobs

X, _ = make_blobs(n_samples=50, centers=4, random_state=42)
X_scaled = StandardScaler().fit_transform(X)

# Build linkage matrix
Z = linkage(X_scaled, method='ward')

plt.figure(figsize=(14, 6))
dendrogram(Z,
            leaf_rotation=90,
            leaf_font_size=8,
            color_threshold=5)
plt.title('Hierarchical Clustering Dendrogram')
plt.xlabel('Sample Index')
plt.ylabel('Distance')
plt.tight_layout()
plt.show()

```

Agglomerative Clustering

```

from sklearn.cluster import AgglomerativeClustering

agg = AgglomerativeClustering(
    n_clusters=4,

```

```

        linkage='ward',          # 'ward', 'complete', 'average', 'single'
        metric='euclidean'      # 'euclidean', 'manhattan', 'cosine'
    )
    labels = agg.fit_predict(X_scaled)

    plt.figure(figsize=(8, 5))
    plt.scatter(X_scaled[:, 0], X_scaled[:, 1], c=labels, cmap='tab10', s=50,
                alpha=0.8)
    plt.title('Agglomerative Clustering')
    plt.tight_layout()
    plt.show()

    print(f"Silhouette Score: {silhouette_score(X_scaled, labels):.4f}")

```

4. Practical Projects

4.1 Document Clustering with TF-IDF + K-Means

```

from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.cluster import KMeans
from sklearn.decomposition import PCA

# Sample documents
documents = [
    "machine learning neural networks deep learning",
    "supervised learning classification regression",
    "python programming data structures algorithms",
    "web development javascript html css",
    "database SQL queries relational tables",
    "unsupervised clustering dimensionality reduction",
    "frontend react vue angular components",
    "data science statistics probability distributions",
    "backend api REST microservices",
    "natural language processing text mining",
]

# TF-IDF
vectorizer = TfidfVectorizer(stop_words='english')
X_tfidf = vectorizer.fit_transform(documents)

# Cluster
km = KMeans(n_clusters=3, n_init=10, random_state=42)
labels = km.fit_predict(X_tfidf)

# Visualize with PCA
pca = PCA(n_components=2)
X_2d = pca.fit_transform(X_tfidf.toarray())

plt.figure(figsize=(10, 6))
for i, (x, y, doc, label) in enumerate(zip(X_2d[:, 0], X_2d[:, 1],
documents, labels)):
    plt.scatter(x, y, c=f'C{label}', s=100)
    plt.annotate(doc[:25] + '...', (x, y), fontsize=7, ha='center',
va='bottom')
plt.title('Document Clustering (TF-IDF + K-Means)')
plt.tight_layout()
plt.show()

```

5. Best Practices & Tips

Preprocessing Checklist

```
# ✅ Always scale features before clustering or PCA
scaler = StandardScaler() # Mean=0, Std=1 (best for most cases)
# or
scaler = MinMaxScaler() # Scale to [0,1] (good if no outliers)

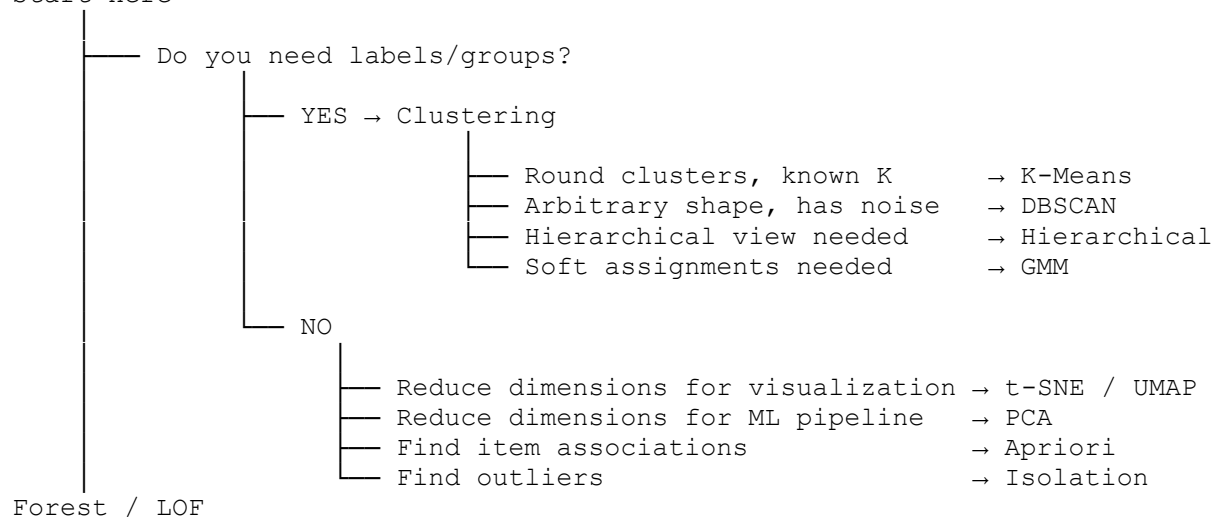
# ✅ Handle missing values
from sklearn.impute import SimpleImputer
imputer = SimpleImputer(strategy='median')
X_imputed = imputer.fit_transform(X)

# ✅ Remove outliers before clustering (use IQR or z-score)
from scipy import stats
z_scores = np.abs(stats.zscore(X))
X_clean = X[(z_scores < 3).all(axis=1)]

# ✅ Encode categorical variables
from sklearn.preprocessing import OneHotEncoder
enc = OneHotEncoder(sparse_output=False, handle_unknown='ignore')
X_encoded = enc.fit_transform(X_cat)
```

Algorithm Selection Guide

Start Here



Common Pitfalls

Pitfall	Solution
Forgetting to scale	Always use <code>StandardScaler</code> before clustering or PCA
Choosing K blindly	Use Elbow + Silhouette methods together
Wrong metric	Use cosine similarity for text, Euclidean for continuous numeric data

Quick Reference Card

Algorithm	Best For	Key Parameters	Weakness
K-Means	Large datasets, spherical clusters	<code>n_clusters</code> , <code>init</code>	Sensitive to outliers, assumes spherical